TroopDrop

A Defence Science Project by Team 18

Ryan Ahn, Fawwaz Ahamed, Raghav Sriram, Hongpeng Wei [Victoria School]

1. An overview

On 15 February 1942, Singapore fell under the hands of the Japanese soldiers. The reason? Couple of reasons, but the most significant one was that we had not expected an attack by the Japanese soldiers on the North front. This left our heavily defended South and East fronts to be rendered completely useless. Another reason is the miscommunication of soldiers. When General Percival ordered troops to prepare to fall back to the city if required, some soldiers thought that they were ordered to immediately go back to the city, allowing the Japanese to infiltrate with little to no resistance.

So let us go back to our modern world. We know the mistakes of the past, so we must try and fix it and prevent the recurrence of such costly mistakes. A mistake which caused us 3 years of suffering and pain, 3 years of hunger and fatigue, 3 years of fear. Is there a way for us to prevent such a mistake again? The answer lies in our proposed idea. A way to regulate troops and enemies and to reassure that whatever is going on in the battlefield is in accordance with our plans. A way that will enhance our perception of the battlefield and enhance our chances of success in battles, all with the usage of our AR technology.

1a. Our Idea: TroopDrop

What is TroopDrop? Perhaps you have heard of AirDrop, the piece of close-range wireless transmission technology invented by Apple.Inc, which we named our project after. We were inspired by the speed and versatility of Apple's AirDrop, and the ability for users to simply drop files to other users' iPhones and other Apple devices. But what has that got to do with TroopDrop, or defence science per se?

What is TroopDrop exactly? We devised TroopDrop as an AR crossover with the use of AR-capable viewers to help soldiers on the battlefield find and determine enemy targets and their comrades. Furthermore, TroopDrop would also display on the lens, directions towards the closest medical station or base, or even cover. AirDrop's wireless and user first capabilities led us to want to design TroopDrop as a wireless transmitting program, using satellite and aircraft imaging and VR technologies to bring the battlefield situation to the soldiers, remotely and wirelessly.

2. The purpose of TroopDrop

So what does TroopDrop aim to achieve? Let us agree on a point, war is messy -- there is a lot of noise and chaos. There is a need for a lot of surrounding awareness in order to plan and guide our troops and come out the victor. TroopDrop is able to detect the locations of both enemies and our own troops, as well as key locations such as our own army base and the locations of the soldiers' missions.

For generals, this makes it easier for them to come up with plans as well as to gain insights into the opponent's plan, so that we can launch a counter attack. This will also allow generals to detect if something has gone wrong during the attack and recall troops or come up with new ideas swiftly. This allows us to save time as well as to gain information on the battlefield, which is highly essential for any war.

" If you know the enemy and know yourself, you need not fear the result of a hundred battles. If you know yourself but not the enemy, for every victory gained you will also suffer a defeat. If you know neither the enemy nor yourself, you will succumb in every battle." ~Sun Tzu

For army troops, these intelligence reports are indeed necessary for victory in battles. This is the reason many nations and their militaries fork out huge sums of money to gather military intelligence (MIL INT) through satellite imaging and reconnaissance. To solve this issue, we find that the best solution is Open Source Intelligence, or OSINT, making maps with accurate real-time information with the help of civilians. Due to software limitations, we decided to use Cospaces to build an animated version of a battlefield instead of stitching together images from open source maps.

3. Layout & Interface

Fig 3.1 Allied troops marked by blue square

3a. Identifying personnel

Welcome to the jungle! In order to build a fitting interface for even the newest of recruits and would suit the old-timers

well too, we had to consider an interface, or rather, user interface (UI) that was both simple, smooth and intuitive, making it easy to learn for all soldiers. Rather than make soldiers feel like they are in the mysterious jungles of Jumanji where dangers lurk everywhere, we would prefer to hand them a handy AR guide to show them where to walk or travel to.

Fig 3.2 Enemy troops marked by red square

Again, the magic of OSINT. Figure 3.1 and 3.2 are self explanatory, they show the enemy troops and your allies.

3b. Landmark Identification

Fig 3.3 Indication of medical camp

The Server is able to detect major landmarks such as the enemy's base, homebase and the headquarters. The following figure shows the location of the medical base. This makes it much easier for soldiers to know where to fall back to if given orders to do so or to give soldiers a direction to attack as per the commands given by the general.

3c. Medical Aid Needed

Fig 3.4 Medical attention required

When a troop is down, they could easily access a button which will indicate that they require medical attention. This allows medics to go to the location and find the troops with greater ease. Since we are equipping the AR goggles on the troopers with vital health sensors, (ie. Heart rate sensors), we are able to feed back to the server regarding the health of troops. If a trooper is unwell, the server would send a warning to the nearest healthy trooper to attend to the unwell person.

3d. Hazard Warning

Fig 3.5 & 3.6 Warning signs of incoming attacks

Threats to troops such as Bio hazards and enemy tanks will be detected through the Lidar and will be shown on the AR screen, thus alerting the troops of the danger and causing lesser casualties.
Explosions can also be detected by Thermal and Infrared Signature Sensors in aircrafts.

3e. Status Bar

Fig 3.7 Communication status

The status bars show the communication status with the server, the status of the location system and the multidirectional bearing device.
The Location System involves Cell Tower Triangulation. Each AR client is a mobile reference point. Using triangulation methods we are able to lock and locate an AR goggle user relative to the server. This allows us to not use a common location protocol for security purposes.

The layout of the program should be simple and easy to control, rather fitting for the VR headsets that the soldiers would be using to view it.

TroopDrop

A Defence Science Project by Team 18

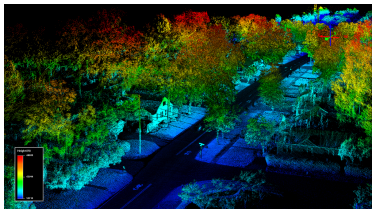
Ryan Ahn, Fawwaz Ahamed, Raghav Sriram, Hongpeng Wei [Victoria School]

Fig 3.8 Whole Layout

4. Efficiency (Real-Time)

We know that efficiency matters, especially more so in times of war. Whatever weapon, plan or directions need to be as efficient and clear as possible -- so we designed TroopDrop with such a thought. TroopDrop refreshes the surroundings and obstacles along with real-time updates on the positioning of friendly and enemy soldiers, while acting as a remote instruction distributor for the generals at war.

5. How Data is collected



Data is collected via imaging from aeroplanes, as well as the cameras and LiDar Sensors on the soldiers' AR Goggles and Tanks to accurately position them. This allows the program to give the soldiers an aerial view of the location that they are in, as well as position the soldiers to direct them to the closest target, base or medical station.

Aerial imaging also allows for more accurate data from a bird's eye view. Furthermore, it allows the soldiers to detect and respond to aerial threats. This will especially benefit crew from artillery and ground-to-air combat squadrons whose task is to shoot down enemy aircraft.

The accurate positioning of friendly and enemy troops as well as directions will further benefit the soldiers fighting in jungle environments, as it may be difficult to navigate in jungles and soldiers may easily get lost.

On the ground level, for tank operators as well as soldiers, their headset will be equipped with lidar sensors and cameras. These Lidar sensors use light detection and ranging to measure the exact distance of an object on the earth's surface. Through these two pieces of equipment, we

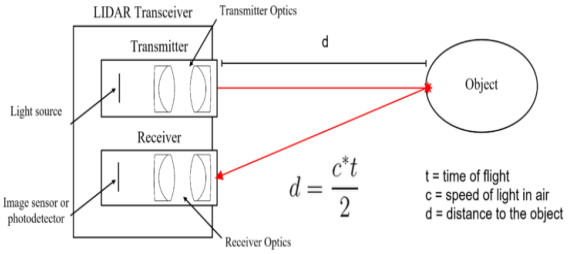


Figure 5.1 Diagram showing how Lidar Sensors work
Picture source:
https://medium.com/swlh/lidar-the-eyes-of-an-autonomous-vehicle-82c6252d1101

are able to sense enemy troops and locate them on the minimap.

Aircraft and helicopters will be tasked with scouting the area and giving an overview of the battlefield. They will use Infrared and thermal sensors as well as cameras to do so.

The data provided by these two sources will be directed to a central server, and the image will be processed under machine learning to detect troops and enemies. This information will then be sent to infantry troops as an AR overlay map in which they can check with a single click.

A. Lidar Sensors

Lidar sensors would be used for two main components in our project, air and on land. Our Lidar sensors would use theLidar sensor processing chip developed by Lincoln Laboratory, which is able to detect more than 16 384 pixels.

Ryan Ahn, Fawwaz Ahamed, Raghav Sriram, Hongpeng Wei [Victoria School]

Figure 5.2 The new Lidar chip and the image it procured
Source: Space shuttle Atlantis

This chip also has the advantage of using indium gallium arsenide (InGaAs). This allows it to be safe when ramped up to high power levels. It also can capture a wide area in a single image. A pass by a business jet at 3,000 meters over Port-au-Prince had captured instantaneous snapshots of 600-meter squares of the city at resolutions of 30 centimetres, and was able to display the precise height of rubble strewn in city streets. This will be placed on soldiers' headsets in order to identify enemies.

For aircrafts, we use these lidar sensors to track down biological weapons , namely poison gas. We plan to use Ultraviolet Laser Induced Fluorescence (UV-LIF) technology.

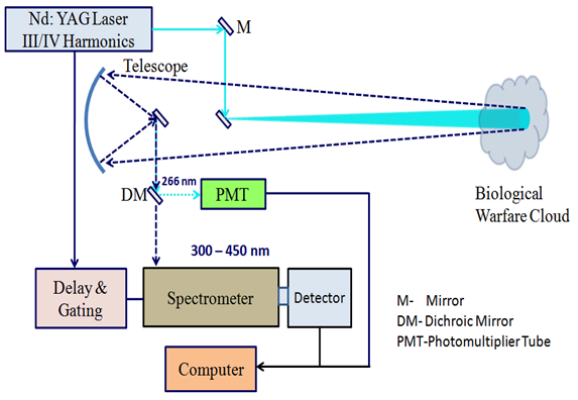


Figure 5.3 Schematic of UV Lidar System

However, we can also consider trying to develop other techniques, more so using Raman Spectroscopy. Raman Spectroscopy uses inelastic scattering of photons and interacts with molecular vibrations, phonons or other excitations in the system. This causes lazer photos to be shifted up and down. The shift in energy will be tracked and give information on the surrounding area. This technology has yet to be implemented on a huger scale, but has been proven effective in identifying vibrational frequencies of chemicals under small samples.

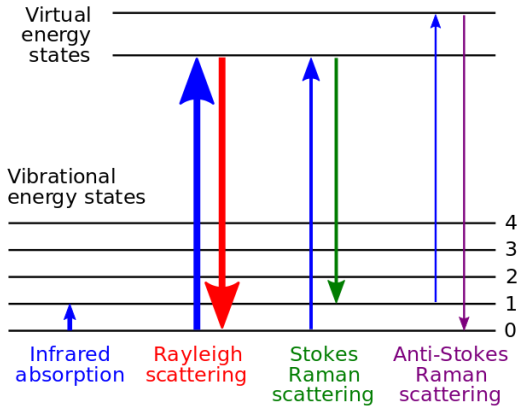


Figure 5.4 Energy level diagram shown in Raman spectra
Source: https://en.wikipedia.org/wiki/Raman_spectroscopy

Lidar sensors have proven to be very useful in detecting biological attacks. In 1994, the Long-Range Biological Standoff Detection System (LR-BSDS) was developed for the U.S. Army to provide a standoff warning for biological attacks. It had detection ranges up to 30km(50km for the objective system) to track aerosol clouds. Our Lilac sensors will be able to scan 30km in approximately 0.20020 milliseconds.

B. Server Management

The data obtained by the sensors will be sent through end-to-end encryption. End-to-end encryption will be done as follows. Let us call the infantry soldier's sensor to be Alice and the AR map that we will be sending Bob. Alice will use Bob's public key to encrypt the message. The encrypted message will get sent to the server and back to Bob. Bob will then use his private key to encrypt the message and get Alice's message.

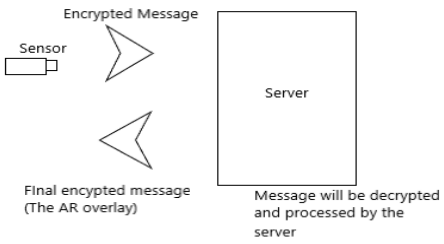


Fig 5.5 End-to-end encryption

After the message is received and decrypted by the computer, machine learning will be used to detect enemies (either camouflaged or exposed), biohazard zones, explosion sites, our own troops as well as medical bases from both the enemy and our own.

Some of the algorithms used will include the following processes:

Supervised Machine Learning The machine is trained based on a predefined set of pictures
Unsupervised Machine Learning The machine is tested based on samples that it has not seen before
Reinforcement Learning The machine will come up with a percentage chance that the object it is given falls into the group it identifies.

Our program will have to incorporate all three components. The following is a sample of a basic ML code which can be used. A step-by-step guide will also be provided. The code is made in Python.

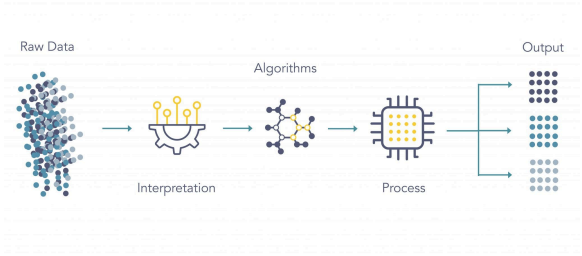


Figure 5.6 Machine Learning process
Source:
https://www.logpoint.com/en/blog/explained-siemply-machine-learning/

Firstly, we have to import the tools which will be used in our Machine Learning.

```
*Following code will not work unless correct repository is provided
import os
import zipfile
import tensorflow as tf
from tensorflow.keras.optimizers import RMSprop
from tensorflow.keras.preprocessing.image import ImageDataGenerator
# need make 2 files: one containing our images and 1 more to test our codes
no_enemies_present = '/content/no_enemies_present'
# one more here for validation
enemies_present = '/content/enemies_present'
```

The following is to set up our two sets of images, namely; no_enemies_present and enemies_present. Each of them contain our training datasets

A validation file is set up in order to test that our robots are able to differentiate images which they have not seen before. The validation file contains a set of 10 images which differ from the training set.

```
#giving directionaries to our data
train_dir = os.path.join(enemies_present, '/content/enemies_present')
validation_dir = os.path.join(validation_images, '/content/validation_images')
# Directory with our enemies_present pictures
enemies_present = os.path.join(train_dir, 'enemies_present')
# Directory with our no_enemies_present pictures
train_no_enemies_present_dir = os.path.join(train_dir, 'no_enemies_present')
# Directory with our validation_no_enemies_present pictures
validtion_no_enemies_present_dir = os.path.join(validation_dir, 'validtion_no_enemies_present')
# Directory with our validation_enemies_present pictures
validation_enemies_present_dir = os.path.join(validation_dir,'validation_enemies_present')
```

The following is done to ensure a standard picture to input into our machine learning program:

Resize images to a 4 x 4

Setting up of matplotlib in order to read 4 x 4 images

```
%matplotlib inline
import matplotlib.pyplot as plt
import matplotlib.image as mpimg
# Parameters for our graph; we'll output images in a 4x4 configuration
nrows = 4
ncols = 4
# Index for iterating over images
pic_index = 0
```

TroopDrop

A Defence Science Project by Team 18

Ryan Ahn, Fawwaz Ahamed, Raghav Sriram, Hongpeng Wei [Victoria School]

The building of a “teacher” bot

Sequential: That defines a SEQUENCE of layers in the neural network

Flatten: Remember earlier where our images were a square, when you printed them out? Flatten just takes that square and turns it into a 1 dimensional set.

Dense: Adds a layer of neurons. Each layer of neurons needs an activation function to tell them what to do. There's lots of options, but just use these for now.

Relu: It effectively means "If X>0 return X, else return 0" -- it only passes values 0 or greater to the next layer in the network.

Softmax: takes a set of values, and effectively picks the biggest one, so, for example, if the output of the last layer looks like [0.1, 0.1, 0.05, 0.1, 9.5, 0.1, 0.05, 0.05, 0.05], it saves you from fishing through it looking for the biggest value, and turns it into [0,0,0,0,1,0,0,0,0] -- The goal is to save a lot of coding

Dropout: This ensures that half of the neurons will be disconnected. This prevents overtraining

```
model = tf.keras.models.Sequential([
                tf.keras.layers.Conv2D(32,    (3,3),
activation='relu', input_shape=(150, 150, 3)),
    tf.keras.layers.MaxPooling2D(2, 2),
                tf.keras.layers.Conv2D(64,    (3,3),
activation='relu'),
    tf.keras.layers.MaxPooling2D(2,2),
                tf.keras.layers.Conv2D(128,    (3,3),
activation='relu'),
    tf.keras.layers.MaxPooling2D(2,2),
                tf.keras.layers.Conv2D(128,    (3,3),
activation='relu'),
    tf.keras.layers.MaxPooling2D(2,2),
    tf.keras.layers.Dropout(0.5),
    tf.keras.layers.Flatten(),
    tf.keras.layers.Dense(512, activation='relu'),
    tf.keras.layers.Dropout(0.5),
    tf.keras.layers.Dense(1, activation='sigmoid')
])
```

Setting up of teacher bot

The teacher bot will test the robots how close they are to the actual objects, that is, whether it is an enemy or not.

The teacher bot uses binary_crossentropy as a method of gauging if the bot's prediction is close to the actual answer. The teacher bot uses the accuracy of each epoch to further develop the neural network of each epoch.

```
from tensorflow.keras.optimizers import RMSprop
model.compile(loss='binary_crossentropy',
              optimizer=RMSprop(lr=0.001),
              metrics=['accuracy'])
```

Setting up of training datasets

The following code takes the sample data and shifts it based on its rotation_range, as well as its height, width, shear and zoom. This is to ensure there is more variance in the training datasets provided, in turn improving accuracy of prediction.

This also sets up the sets of training data provided to each bot. train_generator ensures a supply of training sets while batch_size tells us the size of our training sample, in this case set to 66.

It is important to note that since we used loss = 'binary_crossentopy' in our teacher bot, we need to use binary labels . we denote this by setting class_mode = 'binary' .

```
# preprocessing of data
from    tensorflow.keras.preprocessing.image    import
ImageDataGenerator
train_datagen = ImageDataGenerator(rescale=1./255)
test_datagen = ImageDataGenerator(rescale=1./255)
# All images will be rescaled by 1./255
train_datagen = ImageDataGenerator(
      rescale=1./255,
      rotation_range=40,
      width_shift_range=0.2,
      height_shift_range=0.2,
      shear_range=0.2,
      zoom_range=0.2,
      horizontal_flip=True,
      fill_mode='nearest')
#  Flow  training  images  in  batches  of  20  using
train_datagen generator
train_generator = train_datagen.flow_from_directory(
         train_dir,   # This is the source directory
for training images
        target_size=(150, 150),  # All images will be
resized to 150x150
        batch_size=66,
         # Since we use binary_crossentropy loss, we
need binary labels
        class_mode='binary')
#  Flow  validation  images  in  batches  of  20  using
test_datagen generator
validation_generator                                  =
test_datagen.flow_from_directory(
        validation_dir,
        target_size=(150, 150),
        batch_size=10,
        class_mode='binary')
```

Initiating Machine Learning

```
history = model.fit(
      train_generator,
      steps_per_epoch=2,
      epochs=10,
validation_data=validation_generator,
      validation_steps=2,
      verbose=2)
```

Testing the code

```
import numpy as np
from google.colab import files
from keras.preprocessing import image
uploaded = files.upload()
for fn in uploaded.keys():
  # predicting images
  path = '/content/' + fn
  img = image.load_img(path, target_size=(150, 150))
  x = image.img_to_array(img)
  x = x / 255
  x = np.expand_dims(x, axis=0)
  images = np.vstack([x])
  classes = model.predict(images, batch_size=10)
  print(classes[0])
  if classes[0]>0.5:
    print(fn + " enemy present")
  else:
    print(fn + " enemy absent")
```

The final code testing is not needed, but it sets up a reassurance that the code works.

C. Thermal sensors

Infrared is an electromagnetic radiation which has wavelengths much longer than the visible eye. Infrared cameras are able to track humans through their heat radiation. Humans would radiate wave lengths about 10µm, which can be detected by infrared cameras.

These cameras will be placed on our airplanes and will have a role in detecting our own troops. For our project, we plan on using Forward-Looking Infrared. It is a demographic camera which can sense thermal radiation and can detect wavelengths from 3 to 12µm. More specifically, we can use the AN/AAQ-26. The AN/AAQ-26 has a range of up to 50km, and avoids the disadvantage of active sensors such as night vision or radar. The system incorporates a focal plane array with 480 x 640 detector elements, and the total system weighs less than 90 pounds.

D. Antennas

We plan to input antennas into the AR headsets. This is done in order to find the location of each of our allied soldiers. To improve security and the risk of getting hacked, our system will use triangulation in order to find the exact locations of each soldier.

Triangulation is a method which involves 3 soldiers, to make a triangle with each soldier at the edges and a soldier in the middle of the triangle. The soldier in the middle will send out a signal towards the three other soldiers at the corner of the triangle. Through measuring how long the signal takes to reach each corner, we can determine the position of the soldier in the middle of the triangle.

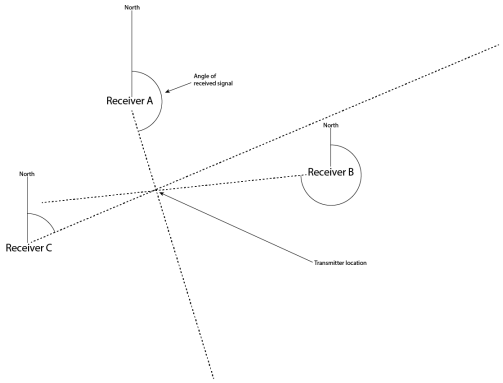


Fig 5.7 Triangulation method to find exact location of soldier
Source:
https://hackaday.io/project/25995-bloodhound-autonomous-ra
cation-drone/log/63866-radio-direction-finding-techniques

No transmitting device can transmit without signal disruptions. As such, we can see that this is a problem.

Hence, it was important that we solve this problem with TroopDrop. Each client AR Screen is a repeater that repeats and propagates the signals sent by and to the server. This allows for a much larger range of data transmission from and to the server. This is can be achieved since we can have a chain of signal connectors so as to allow signal traffic from and to a Client AR screen far away from the server

A repeater (also known as amplifier, or signal booster) is a device used to improve Signal reception in an outdoor environment. A repeater is typically made up of three primary components: A "donor" antenna - also called a "reception" antenna, a bi-directional signal amplifier and one or more rebroadcast antennas. This is NOT a Signal BOOSTER.

A Signal Booster is designed for hardwired applications whereas a repeater is designed to provide wireless coverage to a specified area. Signal Boosters are typically used in M2M applications while repeaters are used in Distributed

Antenna Systems to provide wireless coverage to a specified area.

6. Further Improvements

We as the creators indeed feel the need for improvement. TroopDrop does have many areas that we can touch up on. For one, TroopDrop is limited to usage on VR/AR devices with cameras, in order to provide for the detection and superimposition of 3D VR directions.

Another limitation would be the program's reliance on aerial and LiDar data, which are expensive. However, we hope to be able to transition to OSINT satellite data to make the process even cheaper and more accessible.

Moving forward, we hope to bring TroopDrop to everyone, not as TroopDrop only for troops and the military, but also as a form of navigational guide for civilians, which would allow this new technology to benefit all.

7. Literature Review

After reading a few articles, we felt that communication between soldiers on a battlefield was too important to make a mistake on, but modern solutions do not offer a complete suite of problem-solving ideas.

TroopDrop was made with the intention of helping soldiers to communicate with each other better, as well as to give a better grasp of what is going on in the battlefield.

This is what made us come up with TroopDrop. Thank you for reading.

With Special Thanks to: Defence Science and Technology Agency (DSTA)

Citations:

1. https://publicintegrity.org/national-security/failure-to-communicate-inside-the-armys-doomed-quest-for-the-perfect-radio/;FAILURE TO COMMUNICATE: INSIDE THE ARMY'S DOOMED QUEST FOR THE 'PERFECT' RADIO
2. https://hackaday.io/project/25995-bloodhound-autonomous-radiolocation-drone/log/63866-radio-direction-finding-techniques; Radio Direction Finding Techniques; Phil Hadley
3. http://emano-db.com/pdf/sensor/207/
4. https://www.raytheon.com/capabilities/products/aaq27; AN/AAQ MWIR STARING SENSOR
5. https://www.logpoint.com/en/blog/explained-siemply-machine-learning/; Explain SIEMPly: Machine Learning
6. https://www.potentiaco.com/what-is-machine-learning-definition-types-applications-and-examples/#:~:text=These%20are%20three%20types%20of,unsupervised%20learning%2C%20and%20reinforcement%20learning.; What is Machine Learning: Definition, Types, Applications and Examples
7. https://www.preveil.com/blog/end-to-end-encryption/; What is end-to-end encryption and how does it work?
8. https://www.toptal.com/machine-learning/machine-learning-theory-an-introductory-primer; An introduction to Machine Learning Theory And Its Application: A visual tutorial with examples
9. https://pioneerlabs.io/insights/the-three-types-of-machine-learning-algorithms/; The three types of Machine Learning algorithms
10. Radar is Cheaper but Autonomous Car Needs Lidar**!**; Eric Esteve
11. The World's Most Powerful 3-D Laser Imager; David Talbot
12. Long Range Biological Standoff Detection System (LR-BSDS)
13. LiDAR: The Eyes of an Autonomous Vehicle | by Vedaant Varshney | The Startup
14. https://web.archive.org/web/20110720141859/http://www.rta.nato.int/pubs/rdp.asp?RDP=RTO-TR-SET-098
15. http://articles.janes.com/notice.html
16. Thomas Luhmann; Stuart Robson; Stephen Kyle; Jan Boehm (27 November 2013). *Close-Range Photogrammetry and 3D Imaging*. De Gruyter. ISBN 978-3-11-030278-3.
17. https://web.archive.org/web/20141004201753/http://www.cerdec.army.mil/inside_cerdec/nvesd/
18. Kinsey, J L (1977). "Laser-Induced Fluorescence". *Annual Review of Physical Chemistry*. **28** (1): 349–372. Bibcode:1977ARPC...28..349K. doi:10.1146/annurev.pc.28.100177.002025. ISSN 0066-426X.

19. Richard W. Solarz; Jeffrey A. Paisner (29 September 1986). *Laser Spectroscopy and its Applications*. CRC Press. pp. 623–. ISBN 978-0-8247-7525-4.